

CORE COMPUTER SCIENCE

Object Oriented Programming

A visual C++ reference on classes, inheritance, polymorphism & memory

- › Class & Object Fundamentals
- › Inheritance — 5 Types
- › Encapsulation & Abstraction
- › Polymorphism (Static & Dynamic)
- › Constructors & Destructors
- › Virtual & Pure Virtual Functions
- › Friend Functions & Namespaces
- › Access Specifiers & Key Notes

01 Introduction

Object-Oriented Programming (OOP) is a methodology for designing programs using classes and objects. It simplifies software development and maintenance through a handful of core concepts covered in this reference.

Class

A **class** is a user-defined data type that bundles properties and functions. It is only a logical representation of data — a class occupies no memory until an object is instantiated from it. For example, *Human Being* is a class: body parts are its properties, and actions performed by those parts are its functions.

Object

An **object** is a run-time entity — an instance of a class. It can represent a person, place, or any other item, and can operate on both data members and member functions.

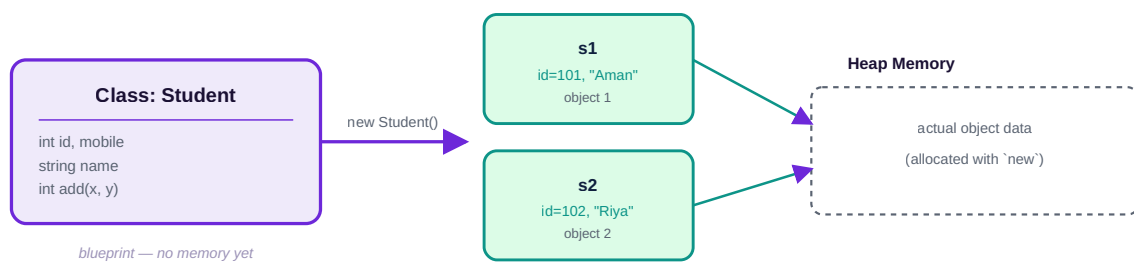


Fig 1 — A class is a blueprint; objects are its run-time instances, each with their own data.

C++ — CLASS DEFINITION

```
class student{
public:
    int id;           // data member
    int mobile;
    string name;
    int add(int x, int y){ // member function
        return x + y;
    }
};
```

C++ — OBJECT CREATION

```
student s = new student();
```

NOTE

When an object is created with **new**, space is allocated on the **heap**, and the starting address is stored in the stack. When created without **new**, no heap space is allocated and the stack variable holds a null value.

02 Inheritance

Inheritance lets one object automatically acquire the properties and behaviors of its parent object, so you can reuse, extend, or modify attributes and behaviors defined in other classes. The class that inherits is the **derived class**; the class being inherited from is the **base class**. The derived class is a specialized version of the base class.

C++ — SYNTAX

```
class derived_class : visibility-mode base_class;  
// visibility-mode = private | protected | public
```

Types of Inheritance

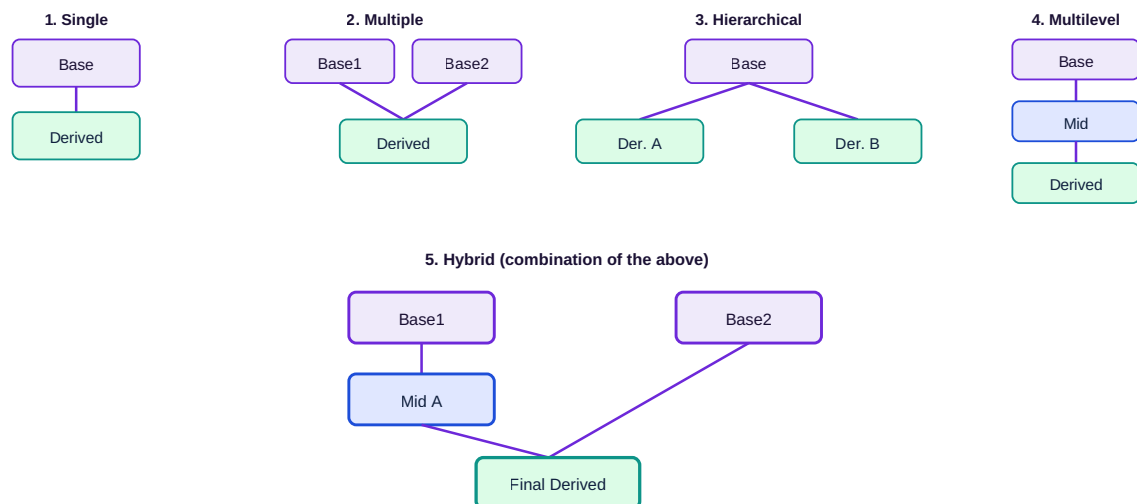


Fig 2 — The five types of inheritance in C++: single, multiple, hierarchical, multilevel and hybrid.

Single

One class inherits from exactly one base class.

Multiple

A derived class inherits attributes from two or more base classes.

Hierarchical

More than one class is derived from a single base class.

Multilevel

A class is derived from another class that is itself derived — a chain.

HYBRID

A combination of simple (single), multiple, and hierarchical inheritance in the same design.

03 Encapsulation & Abstraction

Encapsulation

Encapsulation combines data and functions into a single unit called a class. Data is not accessed directly — it is accessed only through the class's own functions. Attributes are kept **private**, with public *getter/setter* methods to manipulate them. This makes **data hiding** possible (restricting access to members via private/protected, reducing unwanted dependency effects).

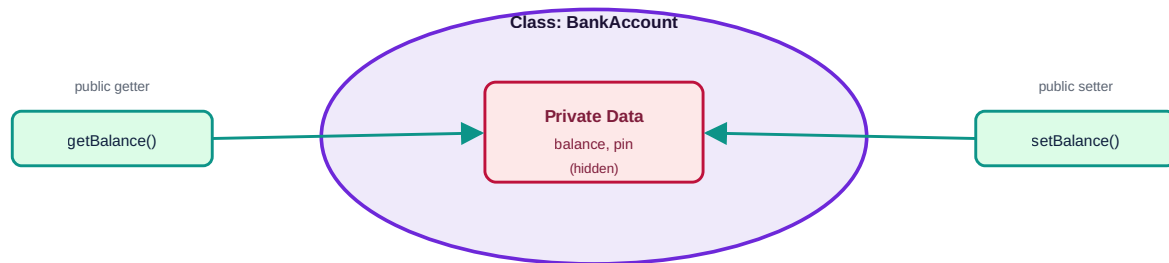


Fig 3 — Private data stays hidden inside the class; the outside world can only reach it through public getter/setter methods.

Abstraction

Abstraction reduces a real-life problem to an abstract model by keeping essential properties and operations while hiding unnecessary details — producing a standard, reusable solution for similar problems.

DATA BINDING

The process of binding an application's UI to its business logic, so any change in the logic is directly reflected in the UI.

04 Polymorphism IMPORTANT

Polymorphism is the ability to present the same interface for differing underlying forms (data types) — *Poly* = many, *morphism* = forms. For example, a point, circle, and rectangle each need different underlying data, yet can share a common "shape" interface.

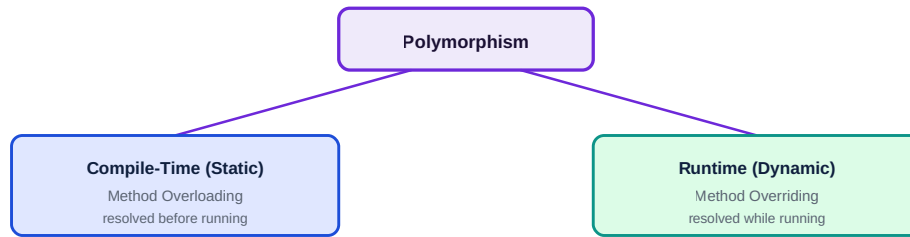


Fig 4 — The two types of polymorphism in C++.

Compile-Time Polymorphism — Method Overloading

Method overloading allows more than one function with the same name but different functionality. Overloading can vary by: (1) return type, (2) parameter types, or (3) number of parameters.

C++ — METHOD OVERLOADING

```
class Add {
public:
    int add(int a, int b){ return (a + b); }
    int add(int a, int b, int c){ return (a + b + c); }
};

int main(){
    Add obj;
    int res1 = obj.add(2,3);           // picks 2-arg version
    int res2 = obj.add(2,3,4);         // picks 3-arg version
    cout << res1 << " " << res2;
}
```

OUTPUT

5 9

Runtime Polymorphism — Method Overriding

Function overriding occurs when a child class redefines a method already present in its parent. Both classes have the same function with different definitions; the version that runs is determined at runtime, based on the actual object type behind a base-class pointer.

C++ — METHOD OVERRIDING

```
class Base_class{
public:
    virtual void show(){ cout << "base"; }
};
class Derived_class : public Base_class{
public:
    void show(){ cout << "derived"; }
};

int main(){
    Base_class* b;
    Derived_class d;
    b = &d;
    b->show();    // prints derived class version
}
```

OUTPUT

derived

05 Constructor & Destructor

A **constructor** is a special method invoked automatically when an object is created, generally used to initialize data members. It shares the class's name. A **destructor** does the opposite — it destructs an object automatically, is defined once per class, and is named like the class prefixed with a tilde (~).

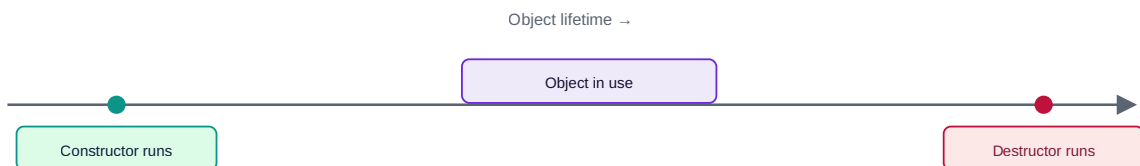


Fig 5 — Constructor fires at creation, destructor fires automatically at the end of an object's life.

Types of Constructors

Default

Takes no arguments; invoked automatically at object creation.

Parameterized

Accepts parameters, letting distinct objects get distinct initial values.

Copy

Initializes an object from another object of the same type — default or user-defined.

C++ — PARAMETERIZED & COPY CONSTRUCTOR

```
class go {
public:
    int x;
    go(int a){ x = a; }           // parameterized constructor
    go(go &i){ x = i.x; }         // copy constructor
};

int main(){
    go a1(20); // parameterized
    go a2(a1); // copy
    cout << a2.x;
}
```

OUTPUT

20

C++ — CONSTRUCTOR / DESTRUCTOR ORDER

```
class A{
public:
    A(){ cout << "Constructor in use" << endl; }
    ~A(){ cout << "Destructor in use" << endl; }
};

int main(){
    A a; A b;
}
```

OUTPUT

Constructor in use Constructor in use Destructor in use Destructor in use

06 'this' Pointer & Friend Function

'this' Pointer

this refers to the current instance of the class. Three common uses:

1. Pass the current object as a parameter to another method.
2. Refer to the current class's instance variables.
3. Declare indexers.

C++ — 'THIS' POINTER

```
struct node{
    int data;
    node *next;
    node(int x){
        this->data = x;
        this->next = NULL;
    }
};
```

Friend Function IMPORTANT

A **friend function** can access a class's private and protected members even though it is not a member of the class itself — it must, however, be listed inside the class definition. It accesses private data using an object name and the dot operator, and it takes objects as arguments.

C++ — FRIEND FUNCTION

```
class A{
    int a = 2, b = 4;
public:
    friend int mul(A k){ return (k.a * k.b); }
};
int main(){
    A obj;
    int res = mul(obj);
    cout << res;
}
```

OUTPUT

8

AGGREGATION

A process where one class defines another class as any entity reference — a way to reuse a class, representing a HAS-A relationship.

07 Virtual & Pure Virtual Functions IMPORTANT

A **virtual function** is a base-class member function that can be redefined by a derived class. When called through a base-class pointer, C++ decides at runtime which version to run based on the actual object type — this enables dynamic dispatch.

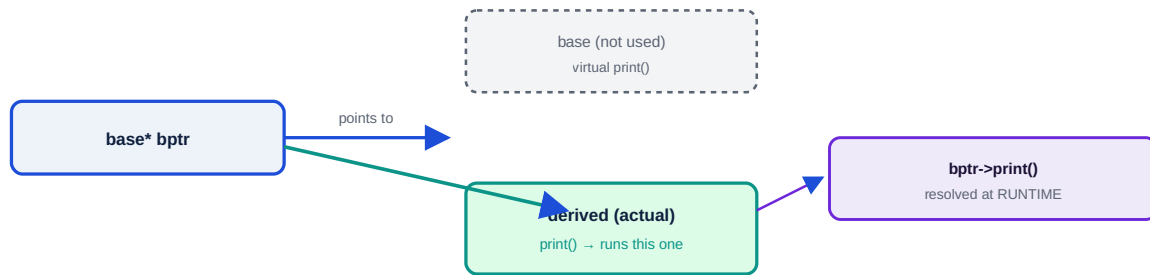


Fig 6 — A base pointer that actually points to a derived object calls the derived class's virtual function.

C++ — VIRTUAL FUNCTION

```

class base {
public:
    virtual void print(){ cout << "print base class"; }
    void show(){ cout << "show base class"; }
};
class derived : public base {
public:
    void print(){ cout << "print derived class"; }
    void show(){ cout << "show derived class"; }
};
int main(){
    base* bptr; derived d; bptr = &d;
    bptr->print(); // virtual - bound at runtime
    bptr->show(); // non-virtual - bound at compile time
}
  
```

OUTPUT

print derived class (impact of virtual function) show base class

KEY POINTS

Virtual functions cannot be static. A class may have a virtual destructor, but never a virtual constructor.

Pure Virtual Function & Abstract Classes

A **pure virtual function** has no definition in the base class — it is only a placeholder, written as `virtual void display() = 0;`. A class containing one becomes an **abstract base class** and cannot be instantiated directly; its role is to define traits for derived classes and to enable a base pointer for runtime polymorphism. Its implementation must be provided by every derived class.

C++ — ABSTRACT CLASS

```
class Shape{    // abstract class
public:
    virtual void draw()=0;
};
class Rectangle : Shape{
public:
    void draw(){ cout << "Rectangle"; }
};
class Square : Shape{
public:
    void draw(){ cout << "Square"; }
};
```

OUTPUT

Rectangle Square

08 Namespaces

A **namespace** is a logical division of code that prevents naming conflicts. It defines the scope in which identifiers — variables, classes, functions — are declared, removing ambiguity when two things share the same name (e.g. two `add()` functions). C++'s standard namespace is `std`, brought in with `using namespace std`;

C++ — USER-DEFINED NAMESPACE

```
namespace Add {
    int a = 5, b = 5;
    int add() { return (a + b); }
}
int main() {
    int res = Add::add(); // access via scope resolution
    cout << res;
}
```

OUTPUT

10

09 Access Specifiers IMPORTANT

Access specifiers define how members can be reached from outside the class.

Specifier	Same Class	Derived Class	Outside Class
Private	✓	✗	✗

Protected	✓	✓	✗
Public	✓	✓	✓

Private

Accessible only within the same class — never outside it.

Public

Accessible from anywhere in the program.

Protected

Accessible in the class and its child (derived) classes only — common in inheritance.

10 Key Notes

- **delete** releases a single unit of memory; **delete[]** releases an array.
- **Virtual inheritance** ensures only one copy of an object exists, even if it appears more than once in the inheritance hierarchy.

Overloading vs. Overriding

Function / Method Overloading

Multiple versions of the same function name with *different signatures* (different parameters), in the **same class**. Resolved at **compile time** (static binding).

Function Overriding

Same method name, same parameters and return type, redefined in a **child class**. Resolved at **runtime** (dynamic binding).

Operator overloading — a standard operator is redefined to take on a different meaning when applied to instances of a class.

	Overloading	Overriding
Binding	Static (compile-time)	Dynamic (runtime)
Classes involved	Same class	Base & derived class
Signature	Different parameters	Same parameters & return type